Frontend Roadmap — Build It In Phases (as you earn)

> You said you want to "build as I earn". This plan is designed for that: each phase ends with something **working and deployable**, and you can start making money as early as **Phase 3**. Nothing later blocks earlier revenue.

Read `docs/CODE_DEEP_DIVE.md` first if you want to understand the API you'll call.

Table of contents

1. Frontend vs backend, in one analogy
2. React concepts you must know
3. Recommended stack
4. Project structure (monorepo)
5. How the frontend talks to this API
6. The phase plan
7. Per-phase detail
8. Deployment & costs
9. Mentor notes for frontend

1. Frontend vs backend, in one analogy

- **Backend** = the restaurant kitchen + pantry. It owns truth (prices, stock, who you are).
- **Frontend** = the dining room: menus, waiters, the vibe. It **asks** the kitchen for things and shows the results.

Rule that never changes: **the frontend is untrusted.** Anyone can open dev-tools and change what your UI "sends". Never decide prices or permissions in the frontend — the backend already does that for you.

2. React concepts you must know

Concept	One-line meaning	Analogy
Component	A function returning UI (JSX)	A reusable LEGO brick
Props	Inputs passed to a component	Ingredients given to a cook
State (`useState`)	Data that changes over time	A whiteboard you can rewrite
Effect (`useEffect`)	Run code after render (fetch, subscribe)	A "do this when X changes" note
Hooks	Functions starting with `use` that add capability	Power-ups
Routing	Mapping URLs to pages	A building's floor directory
Context	Share state without prop-drilling	A building-wide announcement
Controlled input	Form field whose value lives in state	A form the app owns
Rendering	Turning data into DOM	Printing the menu for today

You do **not** need to master all of React before Phase 1. Learn `useState` , `useEffect` , props, and components first; the rest arrives when you need it.

3. Recommended stack

Web (storefront) + Web (admin):

- **Vite + React + TypeScript** — fast dev server, type safety catches mistakes.
- **React Router** — page navigation.
- **TanStack Query (React Query)** — data fetching, caching, loading/error states. This alone removes 50% of hand-written boilerplate.
- **Zustand** (or React Context) — small global state (cart count, auth token).
- **Tailwind CSS** — style fast with utility classes.
- **react-hook-form + zod** — forms + validation.
- **axios** (or a `fetch` wrapper) — HTTP calls with interceptors.
- **lucide-react** / **sonner** — icons / toasts.

Admin dashboard: same base + **shadcn/ui** (components) and **recharts** (charts).

Mobile (the APK): Expo (React Native) + **expo-router** + **TanStack Query**.

> Reality check: React (web) and React Native (mobile) share **language, patterns, and API calls — not DOM components**. Share `api/` , `types/` , and hooks; write screens separately.

4. Project structure (monorepo)

```
frontend/
├── apps/
│   ├── web/          # storefront (Vite + React)
│   │   ├── src/
│   │   │   ├── pages/        # route-level components
│   │   │   ├── components/   # reusable UI
│   │   │   ├── hooks/        # useCart, useProducts...
│   │   │   ├── lib/          # api client, utils
│   │   │   ├── store/        # auth + cart (zustand)
│   │   │   └── main.tsx
│   ├── admin/        # admin dashboard
│   └── mobile/       # Expo app
└── packages/
    ├── api/          # typed API client (generated from OpenAPI)
    ├── types/        # shared TypeScript types
    └── ui/           # shared components (web)
```

- A **monorepo** keeps everything in one Git repo so `packages/` can be shared.
- Use **pnpm workspaces** + **Turborepo** (or just npm workspaces to start).
- **Start simple:** build `apps/web` alone first. Add `packages/*` and `admin` when the storefront works. Don't over-engineer on day one.

5. How the frontend talks to this API

5.1 Base URL + env

```
# apps/web/.env
VITE_API_URL=http://127.0.0.1:8000
```

And add your dev origin to the backend's `CORS_ORIGINS` (already defaults to `localhost:5173`). **If you see CORS errors, this is the fix.**

5.2 A single API client (do this once)

```
// lib/api.ts
import axios from "axios";

export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
});

// attach the JWT to every request automatically
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("access_token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// if the token expired, refresh once and retry
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401) {
      // call POST /auth/refresh with the refresh_token, store new session, retry
    }
    return Promise.reject(error);
  }
);
```

5.3 Auth flow

1. `POST /auth/login` → store `session.access_token` + `session.refresh_token`.
2. Attach the access token (interceptor above).
3. On `401`, `POST /auth/refresh` to get fresh tokens.
4. `POST /auth/logout` and clear storage on sign-out.

Security note for a junior: `localStorage` is easy but readable by any script (XSS). For a production app prefer **httpOnly cookies** (set by the backend). For now, localStorage is acceptable to learn; know the trade-off.

5.4 Data fetching with React Query

```
export function useProducts(filters) {
  return useQuery({
    queryKey: ["products", filters],
    queryFn: async () => (await api.get("/products/", { params: filters })).data.dat
  });
}
```

- `queryKey` = the cache identity. Change the key, get fresh data.
- React Query gives you `isLoading`, `error`, and caching for free.

5.5 Guest vs logged-in (matches the backend)

- No token needed: products, categories, reviews, delivery locations, tracking.
- Token required: cart, wishlist, checkout, profile, uploads, AI chat.
- **Protected route** component: if no token, redirect to `/login` with a `?next=` back-link.

5.6 Optional: generate a typed client

```
# backend running:
curl http://127.0.0.1:8000/openapi.json -o openapi.json
npx openapi-typescript openapi.json -o packages/types/src/api.d.ts
```

Now your editor autocompletes every endpoint and payload. Huge time-saver.

6. The phase plan

Phase	Goal	Earns money?	Rough effort
0	Setup, tooling, API client, design tokens	No	2–4 days
1	**Public storefront: browse products**	No (but showcaseable)	1–2 weeks
2	Auth: signup/login/profile	No	4–6 days
3	**Cart → Checkout → M-Pesa → Tracking**	**Yes**	1.5–2 weeks
4	Customer account: orders, wishlist, reviews	Yes (retention)	1 week
5	Admin dashboard	Yes (operate the shop)	1.5–2 weeks
6	Mobile app (Expo / APK)	Optional	2–3 weeks
7	Polish: PWA, SEO, analytics, monitoring	Yes (scale)	ongoing

Recommendation: do 0→3, launch, then keep going. That's the fastest path to a working store that can take orders.

7. Per-phase detail

Phase 0 — Foundations (2–4 days)

Goal: a running React app that can call the backend.

- Scaffold: `npm create vite@latest web -- --template react-ts` .
- Install: react-router-dom, @tanstack/react-query, axios, tailwindcss, zustand.
- Build `lib/api.ts` (section 5.2) and a `QueryClientProvider` in `main.tsx` .
- Add a `HealthCheck` component calling `GET /health` and `GET /products/` .
- Pick a design language: colors, spacing, font (Tailwind config). **Test:** the page lists real products from your DB. **Gate:** you can fetch and render data.

Phase 1 — Public storefront (1–2 weeks) 🧱

Goal: a visitor can browse and understand the shop without an account. **Endpoints:**

`GET /products/` (search, category, brand, price, sort), `GET /products/{id}`, `GET /categories/`, `GET /reviews/{product_id}`, `GET /delivery/locations`. **Screens:**

- Home (featured products, categories, WhatsApp CTA).
- Product list with filters + search + sort + pagination/infinite scroll.
- Product detail (gallery, price, stock state, "Add to cart" CTA, reviews). **Test:** deploy to Vercel/Netlify (free) and open it on your phone. **Gate:** a stranger can browse your catalogue on a real URL.

Phase 2 — Auth (4–6 days) 🔐

Goal: accounts work end-to-end. **Endpoints:** `POST /auth/signup`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`, `GET/POST/PATCH /profiles/me`. **Screens:** signup, login, forgot-password, profile edit. **Mechanics:** store tokens, attach them, handle 401→refresh, protected routes. **Test:** log in, refresh the page, stay logged in; token expiry refreshes quietly. **Gate:** users can create accounts and see their profile.

Phase 3 — Cart → Checkout → M-Pesa (1.5–2 weeks) 💰

Goal: take real orders and real money. **Endpoints:** `POST /cart/add`, `GET /cart/`, `PATCH /cart/{item_id}`, `DELETE /cart/{item_id}`, `POST /orders/checkout`, `GET /orders/track/{ref}`, `POST /orders/{order_id}/cancel`. **Screens:** cart, checkout (location + phone/email + payment method), order confirmation, order tracking. **Important UX:** after checkout, the user's phone gets an M-Pesa prompt. Show a clear "Check your phone for the PIN prompt" state and poll `GET /orders/track/{ref}` (or `GET /orders/me`) until status changes to `Paid`. **Senior tip:** never show "Paid" optimistically — the backend webhook decides it. **Gate:** a test order goes Pending PIN → Paid with stock correctly decremented. 🎉 **You can now launch and start selling.**

Phase 4 — Customer account (1 week) 📦

Endpoints: `GET /orders/me`, `GET /orders/track/{ref}`, `POST /orders/{id}/cancel`, wishlist add/get/delete, `POST /reviews/`, `POST /upload/profile-picture`. **Screens:** my orders + detail, wishlist, leave a review, avatar upload. **Gate:** returning customers can manage their purchases.

Phase 5 — Admin dashboard (1.5–2 weeks) 🛠️

Endpoints: everything under `/admin/*` plus `POST /products/` (create with images). Your user must be an admin (`ADMIN_EMAILS` or `profiles.role`). **Screens:** login-as-admin, dashboard summary, products CRUD + image manager, orders (filter/status/cancel), categories, inventory/low-stock, delivery locations, customers, reviews moderation, **leads** (list, transcript, reply, assign, stats). **Gate:** you can run the entire shop from the dashboard without touching the DB.

Phase 6 — Mobile app (2–3 weeks) 📱

Goal: an installable APK.

- `npx create-expo-app mobile`.
- Reuse `packages/api` + `packages/types`; rebuild screens with React Native components. Use `expo-router` for navigation.
- Build APK with **EAS Build** (Expo's cloud). **Gate:** installed APK can browse and (optionally) checkout.

Phase 7 — Polish & growth (ongoing) ✨

- PWA (installable web, offline shell).
- SEO + meta tags + sitemap (helps Google find products).
- Analytics (Plausible/GA) + error monitoring (Sentry).
- Image optimization, skeletons, optimistic updates.
- WhatsApp "Chat with us" button wired to your triage bot.

8. Deployment & costs

Piece	Free start	Paid when scaling
Web storefront	Vercel / Netlify (free tier)	~$0–20/mo
Admin	same host, separate subdomain	—
Backend API	Render / Railway / Fly.io free tier	~$5–7/mo
Database/Auth/Storage	Supabase free tier	~$25/mo Pro
Mobile build	Expo EAS free tier	per-build
Domain	—	~$10/yr

Start free. Upgrade only when you have real traffic/revenue.

9. Mentor notes for frontend

1. **Build the smallest page that works, then improve it.** A working ugly page beats a perfect unwritten one.
2. **Always handle three states:** loading, error, empty. Most beginner UIs break on the empty case.
3. **Keep components small.** If a file is >200 lines, split it.
4. **Server state ≠ client state.** Server data belongs in React Query; UI state (modals, forms) in `useState`.
5. **Don't duplicate backend logic.** Recompute nothing about prices/permissions.
6. **Name query keys carefully** (`["products", {filters}]`) or caches go stale.
7. **Test on a slow connection + a small phone.** That's your real user.
8. **Ship Phase 3 early.** Revenue funds the rest and teaches you what users actually want.
9. **Read your network tab.** It shows the exact request/response — the fastest way to debug the frontend↔backend contract.
10. **Consistency beats cleverness.** Same button styles, same spacing, same words across screens.

Study path: finish Phase 1, then read your own API client and a React Query hook out loud. If you can explain each line, you're ready for Phase 3.

You now have: a documented backend, a test guide, a code walkthrough, and a phased frontend plan. Pick Phase 0 and start small.